

Competitive Programming

Section 23 Editorial: Knapsack And Subset DP

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	4
2	Tool reminder	5
3	A. Budget Selection	6
3.1	Restatement	6
3.2	Constraints and traps	6
3.3	Observations	6
3.4	Sample walkthrough	7
3.5	Algorithm	7
3.6	Correctness proof	7
3.7	Complexity	8
3.8	C++ implementation notes	8
3.9	Python implementation notes	8
3.10	Full C++ solution	8
3.11	Full Python solution	9
3.12	Common wrong answers	9
3.13	Tests that matter	9
4	B. Unlimited Training	10
4.1	Restatement	10
4.2	Constraints and traps	10
4.3	Observations	10
4.4	Sample walkthrough	10
4.5	Algorithm	11
4.6	Correctness proof	11
4.7	Complexity	11
4.8	C++ implementation notes	11
4.9	Python implementation notes	12
4.10	Full C++ solution	12
4.11	Full Python solution	12
4.12	Common wrong answers	13
4.13	Tests that matter	13
5	C. Possible Sums	14
5.1	Restatement	14
5.2	Constraints and traps	14
5.3	Observations	14
5.4	Sample walkthrough	14
5.5	Algorithm	15
5.6	Correctness proof	15
5.7	Complexity	15
5.8	C++ implementation notes	16
5.9	Python implementation notes	16

5.10 Full C++ solution	16
5.11 Full Python solution	17
5.12 Common wrong answers	17
5.13 Tests that matter	17
6 Testing and stress strategy	19
7 What to carry forward	20

1 Editorial overview

Problem	Item rule	DP array	Loop direction
A. Budget Selection	each item at most once	best value by budget	backward
B. Unlimited Training	each type unlimited	best value by budget	forward
C. Possible Sums	each number at most once	reachable sums	backward

The section's central invariant is:

After processing some prefix of items, the DP array describes what is possible using only that prefix.

Loop direction controls whether the current item can see itself.

2 Tool reminder

For a one-dimensional capacity DP:

```
dp[cap] = best answer using processed items with capacity cap
```

For 0/1 items, update `cap` descending:

```
for cap = W down to cost
```

For unlimited items, update `cap` ascending:

```
for cap = cost up to W
```

For subset feasibility:

```
possible[sum] = true/false
```

and use descending sums when each number is usable once.

3 A. Budget Selection

3.1 Restatement

There are n optional resources. Resource i has a cost and a value. You may choose each resource at most once. With total cost at most W , maximize total value.

3.2 Constraints and traps

The budget W is small enough for $O(nW)$, but 2^n subset enumeration is not.

Important traps:

```
Each resource is optional and usable at most once.  
Some resources may cost more than  $W$  and can never be chosen.  
Values can be large, so use long long in C++.  
Backward capacity iteration is required after memory compression.
```

3.3 Observations

With the first i resources and budget cap , the last decision is:

```
skip resource i  
take resource i, if  $cost[i] \leq cap$ 
```

The two-dimensional recurrence is:

```
 $dp[i][cap] = \max(dp[i-1][cap],$   
                   $dp[i-1][cap - cost[i]] + value[i])$ 
```

We compress rows into one array. To preserve the meaning of $dp[i-1]$ while processing item i , capacities must go downward.

3.4 Sample walkthrough

Input:

```
4 7
3 4
4 5
2 3
5 8
```

The best choice is:

```
resource 3: cost 2, value 3
resource 4: cost 5, value 8
```

Total cost 7, total value 11.

3.5 Algorithm

```
dp[0..W] = 0

for each item cost, value:
    for cap from W down to cost:
        dp[cap] = max(dp[cap], dp[cap-cost] + value)

print max(dp)
```

`max(dp)` is used because the problem allows spending at most W , not exactly W . In this implementation `dp[cap]` is monotone for nonnegative values, but `max(dp)` states the intent explicitly.

3.6 Correctness proof

Lemma 1. When processing an item, descending capacity order ensures `dp[cap-cost]` still represents a solution that does not use the current item.

Proof. Since `cap-cost < cap`, and capacities are processed from large to small, `dp[cap-cost]` has not yet been updated during this item's iteration. It is therefore still the value from before the current item was processed. $\square$

Lemma 2. After processing the first i items, `dp[cap]` is the maximum value achievable with capacity at most `cap` using only those i items.

Proof. By induction on i . Initially, no item has been processed, so value 0 is the best for every capacity. For item i , every valid solution either skips it, giving the old `dp[cap]`, or takes it, requiring a valid solution of capacity `cap-cost[i]` from previous items plus `value[i]`. By Lemma 1, the transition reads exactly that previous-item value. Taking the maximum covers both exhaustive cases. $\square$

Theorem. The algorithm prints the maximum total value obtainable within budget W .

Proof. By Lemma 2 after all items are processed, every $dp[cap]$ is optimal for that capacity. The best legal budget is the maximum over $cap \leq W$. $\square$

3.7 Complexity

```
Time:  $O(nW)$   
Memory:  $O(W)$ 
```

3.8 C++ implementation notes

Use `vector<long long>` for `dp`. Skip an item with `cost > W` to avoid an empty or invalid loop.

3.9 Python implementation notes

The descending loop is:

```
for cap in range(W, cost - 1, -1):
```

3.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, W;
    if (!(cin >> n >> W)) return 0;
    vector<long long> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        int cost;
        long long value;
        cin >> cost >> value;
        if (cost > W) continue;
        for (int cap = W; cap >= cost; cap--) {
            dp[cap] = max(dp[cap], dp[cap - cost] + value);
        }
    }
    cout << *max_element(dp.begin(), dp.end()) << '\n';
    return 0;
}
```

3.11 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    w = next(it)
    dp = [0] * (w + 1)
    for _ in range(n):
        cost = next(it)
        value = next(it)
        if cost > w:
            continue
        for cap in range(w, cost - 1, -1):
            cand = dp[cap - cost] + value
            if cand > dp[cap]:
                dp[cap] = cand
    print(max(dp))

if __name__ == "__main__":
    main()
```

3.12 Common wrong answers

Looping cap upward and using the same item multiple times.

Using int for values.

Assuming every item has cost $\leq W$.

Printing an uninitialized impossible state in variants where exact capacity is required.

3.13 Tests that matter

$W = 0 \rightarrow$ answer 0.

All costs greater than $W \rightarrow$ answer 0.

One item with cost c should not be usable twice.

Large values test long long.

4 B. Unlimited Training

4.1 Restatement

There are n drill types. Drill type i has a time cost and a value. You may perform each drill type any number of times. With at most W total minutes, maximize total value.

4.2 Constraints and traps

The budget is small enough for $O(nW)$.

Important traps:

```
The same drill type may be repeated.  
Forward capacity iteration is required to allow repetition.  
If  $W = 0$ , the answer is 0.  
Values can be large after many repetitions.
```

4.3 Observations

For each drill type, after we know a best value for capacity `cap-cost`, taking one more copy of the same drill gives a candidate for `cap`.

Unlike 0/1 knapsack, seeing the current item inside the same item iteration is intended. That is why the capacity loop goes forward.

4.4 Sample walkthrough

Input:

```
3 10  
6 30  
3 14  
4 16
```

Best choice:

```
one copy of cost 6, value 30  
one copy of cost 4, value 16
```

Total value: 46.

4.5 Algorithm

```
dp[0..W] = 0

for each drill type cost, value:
    for cap from cost to W:
        dp[cap] = max(dp[cap], dp[cap-cost] + value)

print max(dp)
```

4.6 Correctness proof

Lemma 1. During the forward loop for a drill type, $dp[cap-cost]$ contains the best value using processed types, including any number of copies of the current type.

Proof. Since $cap-cost < cap$, it was processed earlier in the same forward loop. Therefore it already includes all improvements available with the current drill type at that smaller capacity. $\square$

Lemma 2. After processing the first i drill types, $dp[cap]$ is the maximum value achievable with capacity at most cap using only those types, with unlimited copies.

Proof. By induction on i . Initially, no drill gives value, so all states are 0. For type i , a best solution either uses no copy of this type, leaving the old $dp[cap]$, or uses at least one copy. Removing one copy leaves a valid solution of capacity $cap-cost[i]$ that may still use type i ; by Lemma 1 that value is available as $dp[cap-cost[i]]$. The transition takes the better of these cases. $\square$

Theorem. The algorithm prints the maximum value obtainable within W minutes.

Proof. Lemma 2 after all drill types gives optimal values for every legal capacity. The answer is the best among capacities at most W . $\square$

4.7 Complexity

```
Time:  $O(nW)$ 
Memory:  $O(W)$ 
```

4.8 C++ implementation notes

Use `long long` because value can grow with repeated drills.

4.9 Python implementation notes

The forward loop is:

```
for cap in range(cost, W + 1):
```

4.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, W;
    if (!(cin >> n >> W)) return 0;
    vector<long long> dp(W + 1, 0);
    for (int i = 0; i < n; i++) {
        int cost;
        long long value;
        cin >> cost >> value;
        if (cost > W) continue;
        for (int cap = cost; cap <= W; cap++) {
            dp[cap] = max(dp[cap], dp[cap - cost] + value);
        }
    }
    cout << *max_element(dp.begin(), dp.end()) << '\n';
    return 0;
}
```

4.11 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    w = next(it)
    dp = [0] * (w + 1)
    for _ in range(n):
        cost = next(it)
```

```

    value = next(it)
    if cost > w:
        continue
    for cap in range(cost, w + 1):
        cand = dp[cap - cost] + value
        if cand > dp[cap]:
            dp[cap] = cand
    print(max(dp))

if __name__ == "__main__":
    main()

```

4.12 Common wrong answers

Looping cap downward, which turns the problem into 0/1 knapsack.
 Thinking the sample must use as much budget as possible.
 Using int when repeating a high-value drill many times.

4.13 Tests that matter

One drill type, W multiple of cost -> repeated copies.
 W = 0 -> 0.
 A high-value expensive type plus lower-value repeatable type.
 An item whose cost exceeds W should have no effect.

5 C. Possible Sums

5.1 Restatement

Given n positive integers, each usable at most once, print every positive sum that can be formed by choosing a subset.

5.2 Constraints and traps

The total sum is at most 100000, so subset DP over sums is possible.

Important traps:

```
Each number is usable at most once.  
The empty subset gives sum 0, but 0 is not printed.  
Repeated values are still separate items.  
Backward sum iteration is required.  
Output sums in increasing order.
```

5.3 Observations

Let:

```
possible[s] = whether some subset of processed numbers has sum s
```

Initially:

```
possible[0] = true
```

For each number x , every previously reachable sum s creates a new reachable sum $s+x$. Because the number can be used once, iterate s downward.

5.4 Sample walkthrough

Numbers:

```
4 2 5 2
```

Reachable positive sums:

```
2 4 5 6 7 8 9 11 13
```

There are 9 of them.

5.5 Algorithm

```
total = sum(a)
possible[0] = true
current = 0

for x in a:
    for s from current down to 0:
        if possible[s]:
            possible[s+x] = true
    current += x

answer = all s from 1 to total where possible[s]
print count and list
```

The variable `current` is the largest sum reachable using processed numbers so far, so the loop does not scan unused high sums.

5.6 Correctness proof

Lemma 1. When processing number x , descending order ensures every source sum s used for $s+x$ was reachable before using x .

Proof. Since $s < s+x$, and the loop goes downward, setting `possible[s+x]` cannot later be used as a source for another addition of the same x . $\square$

Lemma 2. After processing the first i numbers, `possible[s]` is true exactly when some subset of those i numbers has sum s .

Proof. By induction. Before processing any number, only the empty subset sum 0 is reachable. For a new number x , every subset either excludes x , keeping an old reachable sum, or includes x , in which case removing x leaves an old reachable sum $s-x$. The transition marks exactly those new sums, and Lemma 1 prevents using x more than once. $\square$

Theorem. The algorithm prints exactly all positive subset sums in increasing order.

Proof. By Lemma 2 after all numbers are processed, `possible[s]` exactly describes reachable subset sums. Scanning s from 1 to `total` prints all positive reachable sums in increasing order. $\square$

5.7 Complexity

```
Time: O(n * total_sum)
Memory: O(total_sum)
```

5.8 C++ implementation notes

Use `vector<char>` for the boolean table. Track `current` to reduce constant factors.

5.9 Python implementation notes

A list of booleans is enough for these constraints. More advanced bitset tricks exist, but they are not required here.

5.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<int> a(n);
    int total = 0;
    for (int &x : a) {
        cin >> x;
        total += x;
    }
    vector<char> possible(total + 1, false);
    possible[0] = true;
    int current = 0;
    for (int x : a) {
        for (int s = current; s >= 0; s--) {
            if (possible[s]) possible[s + x] = true;
        }
        current += x;
    }
    vector<int> ans;
    for (int s = 1; s <= total; s++) {
        if (possible[s]) ans.push_back(s);
    }
    cout << ans.size() << '\n';
    for (int i = 0; i < (int)ans.size(); i++) {
        if (i) cout << ' ';
```

```

        cout << ans[i];
    }
    cout << '\n';
    return 0;
}

```

5.11 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    a = data[1:1 + n]
    total = sum(a)
    possible = [False] * (total + 1)
    possible[0] = True
    current = 0
    for x in a:
        for s in range(current, -1, -1):
            if possible[s]:
                possible[s + x] = True
        current += x
    ans = [i for i in range(1, total + 1) if possible[i]]
    print(len(ans))
    print(" ".join(map(str, ans)))

if __name__ == "__main__":
    main()

```

5.12 Common wrong answers

Looping upward and using the same number multiple times.
 Forgetting possible[0] = true.
 Printing sum 0 even though the statement asks for positive sums.
 Deduplicating equal input numbers incorrectly; equal values are still separate items.

5.13 Tests that matter

Single number -> one positive sum.
All numbers equal -> sums are multiples up to total.
Repeated values with different subsets.
Maximum total sum tests memory and time.

6 Testing and stress strategy

Run:

```
CP_TARGET=solution python3 sections/23_knapsack_subset_dp/check.py
```

Random tests target:

Budget Selection:

costs above W , zero budget, and high values.

Unlimited Training:

cases where repeating one type is optimal.

Possible Sums:

repeated values and dense reachable-sum ranges.

For tiny stress tests:

A: enumerate every subset and compare best value.

B: run a shortest-path style relaxation or recursive brute force for tiny W .

C: enumerate every subset and collect sums for $n \leq 20$.

7 What to carry forward

Knapsack DP is not one algorithm. It is a family of capacity-indexed states.

The key questions are:

What does the capacity mean?

Can each item be used once or many times?

Does the loop direction enforce that rule?

Is capacity small enough, or should another dimension be used?

Those questions return in later sections on bitmask DP, value-based knapsack, and optimization DP.